

AI Web Search Agent

Project Report

1. Project Overview

The AI Web Search Agent is a full-stack intelligent application that combines Large Language Models (LLMs), LangGraph agents, tool calling, and real-time streaming to provide conversational answers enhanced with web search capabilities. The system supports autonomous tool usage, markdown rendering, syntax highlighting, and mathematical equation rendering in a ChatGPT-like interface.

2. Objectives :

- Build an AI-powered conversational assistant.
- Implement autonomous tool usage using LangGraph.
- Integrate web search capabilities.
- Stream responses in real time using Server-Sent Events (SSE).
- Render markdown, code blocks, and mathematical equations.
- Develop a modern and responsive React interface.

3. Technologies Used

Frontend: React.js, Vite, Tailwind CSS, React Markdown

Backend: FastAPI, LangChain, LangGraph

LLM Provider: OpenRouter

Streaming: Server-Sent Events (SSE)

AI Rendering: remark-gfm, rehype-highlight, KaTeX

Other: Axios, CORS Middleware

4. System Architecture

React Frontend → Axios API Layer → FastAPI Backend → LangGraph Agent → LLM + Tools
→ Response Streaming → Frontend Rendering

5. Modules and Features

Frontend Module:

- ChatGPT-style UI
- Real-time streaming responses
- Markdown, code, and math rendering

Backend Module:

- FastAPI REST APIs
- SSE streaming endpoint

- Error handling and CORS support

Agent Module:

- LangGraph workflow orchestration
- Tool calling and autonomous reasoning

Tool Module:

- Web search tool integration
- Calculator/tool execution support

6. LangGraph Workflow

1. User submits query.
2. LangGraph assistant node invokes the LLM.
3. LLM decides whether tools are required.
4. Tool node executes selected tools.
5. Results are returned to the LLM.
6. Final answer is streamed to the frontend.

7. API Endpoints

GET /

POST /chat

POST /stream

8. Streaming Workflow

1. User sends query from React UI.
2. Frontend calls /stream endpoint.
3. FastAPI streams SSE events.
4. Frontend reconstructs SSE messages.
5. Markdown is rendered after stream completion.

9. Challenges Faced

- Handling SSE chunk parsing.
- Rendering markdown during streaming.
- Managing LangGraph tool execution.
- Preventing network errors and incomplete chunk encoding.
- Handling model rate limits.

10. Solutions Implemented

- Implemented semantic chunk buffering.
- Added robust SSE parsing.
- Used ReactMarkdown with syntax highlighting.
- Added KaTeX for math rendering.
- Added error handling and graceful stream termination.

11. Future Scope

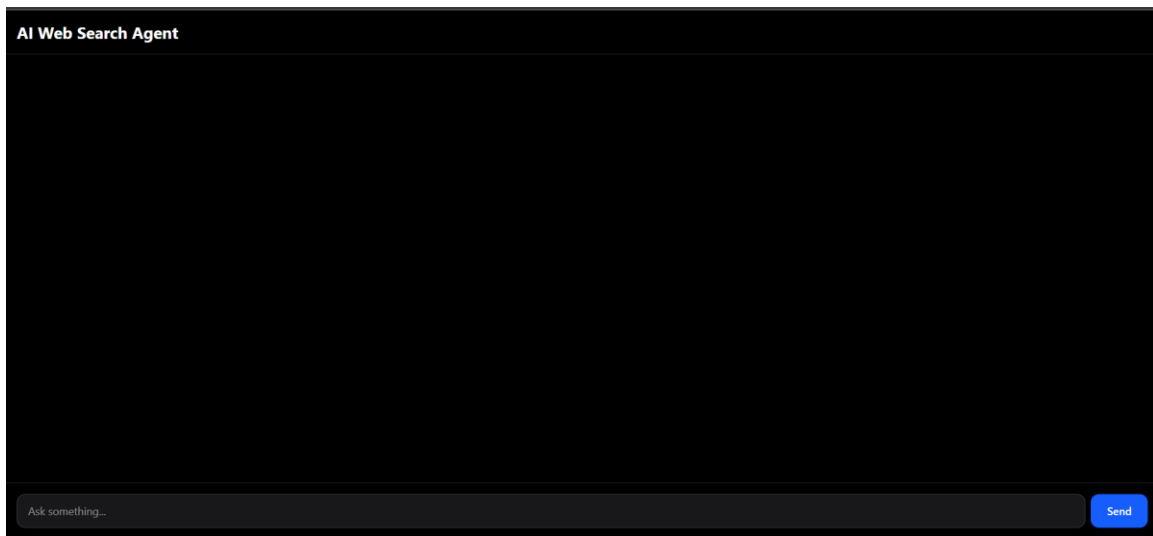
- Multi-agent architecture.
- RAG integration with vector databases.
- Chat history persistence.
- Authentication and user management.
- Voice and image input support.

12. Conclusion

The AI Web Search Agent demonstrates industry-level concepts including agentic AI, LangGraph orchestration, tool calling, streaming architectures, and modern AI frontend rendering. The project reflects practical experience in building real-time AI applications.

13. Screenshots

Screenshot 1: _____



Screenshot 2: _____

AI Web Search Agent

hi

Hello! How can I assist you today?

what is the today weather report of pune in short

Weather in Pune (as of the latest available data)	Parameter	Current Condition
Temperature		~28 °C (82 °F)
Feels Like		~30 °C (86 °F)
Sky		Mostly sunny with a few scattered clouds
Humidity		55%
Wind		12km/h (7 mph) from the southwest
Chance of Rain		0% (dry)
Air Quality (AQI)		45 - Good

Note: This is a short-term snapshot based on the most recent publicly available forecasts. For the most up-to-date and location-specific report, please check a reliable weather service (e.g., Weather.com, AccuWeather, or a local meteorological department).

Screenshot 3: _____

AI Web Search Agent

write hello world code in c++

Here's the classic "Hello, World!" program in C++:
cpp
// hello_world.cpp
#include <iostream>
int main() {
 // Print the message to the console
 std::cout << "Hello, World!" << std::endl;
 return 0; // Indicate successful execution
}
How to compile and run
Save the file as hello_world.cpp.
Compile it with a C++ compiler (e.g., g++ or clang++):
bash
g++ -std=c++17 -Wall -Wextra -O2 -o hello_world hello_world.cpp
Run the resulting executable:
bash
./hello_world
You should see:
Hello, World!

Screenshot 4: _____

AI Web Search Agent

what is answer of 5*(3+4*2)

Here's the classic "Hello, World!" program in C++:
cpp
// hello_world.cpp
#include <iostream>
int main() {
 // Print the message to the console
 std::cout << "Hello, World!" << std::endl;
 return 0; // Indicate successful execution
}
How to compile and run
Save the file as hello_world.cpp.
Compile it with a C++ compiler (e.g., g++ or clang++):
bash
g++ -std=c++17 -Wall -Wextra -O2 -o hello_world hello_world.cpp
Run the resulting executable:
bash
./hello_world
You should see:
Hello, World!

Answer: [5 \times (3 + 4 \times 2) = 5 \times (3 + 8) = 5 \times 11 = \boxed{55}]